

Exercícios — Aula 02 — ArrayList, TAD e custos de operações

Técnicas de Programação - 2026/02

Última atualização: September 2, 2026

Instruções

- Neste handout, **Lista** é um TAD: use as operações abstratas, não detalhes de uma linguagem.
- Simule o estado da lista após cada operação.
- Quando analisar custo, pense no pior caso e na operação dominante.

Exercício 1 — Simulação de operações em lista

Simule o pseudocódigo abaixo. Depois de cada linha numerada, escreva o estado da lista e a saída, se houver.

Algorithm 1: ListaDeTarefas

Result: Executa LISTA-DE-TAREFAS.

Input : none

Output: textos impressos

```
L ← LISTA-VAZIA()
ADICIONAR-FIM(L, "ler")
ADICIONAR-FIM(L, "simular")
ADICIONAR-FIM(L, "testar")
IMPRIMIR(OBTER(L, 1))

ALTERAR(L, 1, "desenhartabela")
ADICIONAR-FIM(L, "entregar")
REMOVER(L, 0)

if CONTEM(L, "testar") then
| IMPRIMIR("ainda faltatestar")
end
else
| IMPRIMIR("testeremovido")
end
IMPRIMIR(TAMANHO(L))
```

passo	operação	estado de L após o passo	saída
1	ADICIONAR-FIM(L, "ler")		
2	ADICIONAR-FIM(L, "simular")		
3	ADICIONAR-FIM(L, "testar")		
4	IMPRIMIR(OBTER(L, 1))		
5	ALTERAR(L, 1, "desenhar tabela")		
6	ADICIONAR-FIM(L, "entregar")		
7	REMOVER(L, 0)		
8	CONTEM(L, "testar")		
9	IMPRIMIR(TAMANHO(L))		

Exercício 2 — Custo das operações

Classifique o custo esperado de cada operação em uma lista baseada em array como **constante**, **linear** ou **depende da posição**.

Operação	Custo	Justificativa curta
OBTER(L, i)		
ALTERAR(L, i, valor)		
ADICIONAR(L, valor) no fim		
INSERIR(L, 0, valor)		
REMOVER(L, 0)		
REMOVER(L, TAMANHO(L)-1)		
CONTEM(L, valor)		

Em qual dessas operações pode ser necessário deslocar vários elementos?

Exercício 3 — Filtrar valores maiores que um limite

Complete o pseudocódigo para devolver uma nova lista com os valores maiores que **limite**, preservando a ordem original.

Algorithm 2: MaioresQue

Result: Executa MAIORES-QUE.

Input : list values, number limite

Output: list resposta

```
resposta ← LISTA-VAZIA()
for i ← 0 to TAMANHO(valores) − 1 do
    valor ← OBTER(valores, i)
    if _____ then
        _____
    end
end
return resposta
```

Teste com:

- valores = [4, 10, 3, 12, 8], limite = 7;
- valores = [1, 2, 3], limite = 5;
- valores = [], limite = 0.

Exercício 4 — Remoção durante percurso

O algoritmo abaixo tenta remover todas as ocorrências de "ausente" de uma lista. Ele parece razoável, mas possui um erro.

Algorithm 3: RemoveAusentesComErro

Result: Executa REMOVE-AUSENTES-COM-ERRO.**Input :** list nomes**Output:** list nomes alterada

```
for  $i \leftarrow 0$  to  $TAMANHO(nomes) - 1$  do
  if  $OBTER(nomes, i) = \text{"ausente"}$  then
    REMOVE( $nomes, i$ )
  end
end
return  $nomes$ 
```

1. Simule para $nomes = ["Ana", "ausente", "ausente", "Bia"]$.
2. Qual o erro? Quando ele acontece?
3. Reescreva o algoritmo usando as estratégias abaixo e veja qual delas soluciona o problema:
 - percorrer de trás para frente;
 - manter i no mesmo lugar quando uma remoção acontece.

Exercício 5 — Histórico de notas

Escreva pseudocódigo para um TAD `HistoricoNotas` usando uma lista de notas.

Ele deve oferecer as operações:

- `ADICIONAR-NOTA(H, nota);`
- `MEDIA(H);`
- `MAIOR-NOTA(H);`
- `REMOVER-POSICAO(H, indice).`

Onde `H` é uma instância do TAD `HistoricoNotas`. Defina o que cada operação retorna quando a lista está vazia ou quando o índice é inválido.

Exercício 6 — Array fixo ou lista dinâmica

Para cada situação, escolha **array fixo** ou **lista dinâmica** e justifique em uma frase.

Situação	Escolha	Justificativa
Guardar as 4 notas bimestrais de um estudante		
Montar a lista de inscritos enquanto as inscrições chegam		
Guardar 7 temperaturas, uma para cada dia da semana		
Registrar mensagens recebidas até o usuário fechar o sistema		
Percorrer uma coleção sem saber antes quantos itens válidos haverá		